3D-SubG: A 3D Stacked Hybrid Processing Near/In-Memory Accelerator for Subgraph GNNs

Guoxiang Li[1,2], Runnan Xu[1], Ruohang Xu, Yikan Qiu, Renati Tuerhong, Muhan Zhang[1], Le Ye[2], Yufei Ma[1,2*]
[1]Institute for Artificial Intelligence, Peking University, Beijing, China
[2]School of Integrated Circuits, Peking University, Beijing, China

2025 62nd ACM/IEEE Design Automation Conference (DAC) | 979-8-3315-0304-8/25/$31.00 ©2025 IEEE | DOI: 10.1109/DAC63849.2025.11132830

***Abstract*—Subgraph Graph Neural Networks (GNNs) are emerging as a promising approach to enhance GNN expressiveness, but their more complex graph structures with numerous independent and irregular subgraphs pose significant hardware deployment challenges. In this work, we propose 3D-SubG, a 3D stacked hybrid processing-near/in-memory accelerator for subgraph GNNs. With hybrid bonding packaging technology, a logic die is 3D stacked with a DRAM die for highly parallel memory accesses. The logic die employs digital SRAM-based processing-in-memory (PIM) macros to boost computation density and minimize data transfer. We further propose a bit-level non-zero gathering method to exploit graph sparsity for PIM, a workload-balanced mapping strategy for subgraph allocation onto different logic-to-DRAM blocks, and a distributed global pooling approach to reduce inter-block data movements. Experimental results show that 3D-SubG achieves average improvements of 146.11× in performance, 934.18× in area efficiency, and 1171.80× in energy efficiency compared to RTX 3090Ti.**

***Index Terms*—Processing in Memory, Graph Neural Networks, Near-Memory Processing, 3D Stacked Architecture**

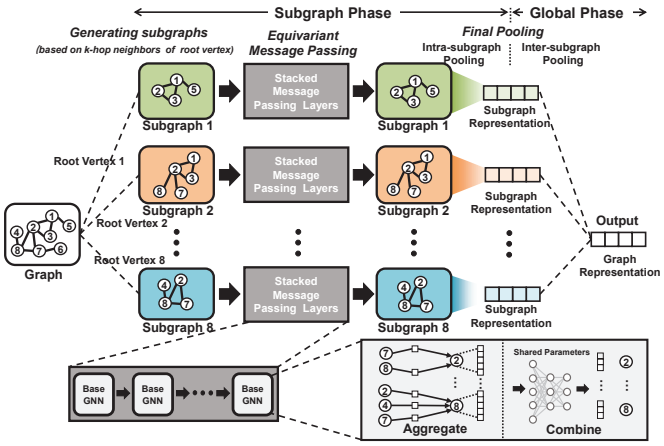

Fig. 1. An illustration of general subgraph GNN architecture.

I. Introduction

Graph Neural Networks (GNNs) have exhibited exceptional capabilities in handling graph-structured data across diverse domains, including graph mining [1], [2], recommendation systems [3], biology [4], chemistry [5], and so on. However, there is a growing consensus that traditional GNNs suffer from limited expressive power [6]–[9], constrained by the first-order Weisfeiler-Leman (1-WL) test [10], which is a common measure of GNN expressiveness. Studies indicate that GNNs equivalent to the 1-WL test fall short in capturing essential structural concepts, like motif counting (e.g., cycles, triangles) [9], [11], [12], which are proven to be pivotal in fields such as bioinformatics and chemoinformatics [13].

Recently, subgraph GNNs [9], [14]–[17], as illustrated in Fig. 1, are emerging to surpass the expressive power of traditional GNNs, approaching the level of 3-WL [9], [14], [18]. A general subgraph GNN operates in two phases: the subgraph phase, which confines computations to individual subgraphs, and the global phase, which processes all subgraphs. By decomposing a graph into multiple subgraphs processed by independent base GNNs, subgraph GNNs boost expressive power. However, they demand significantly higher hardware complexity due to the need to process numerous subgraphs with vertices often vastly exceeding the original graphs by orders of magnitude, as shown in Table II. It is worth noting that the same vertices shared among different subgraphs cannot reuse vertex feature vectors, because after message passing, the feature values of the same vertex differ across subgraphs.

Subgraph GNNs pose greater challenges for hardware deployment. Both CPUs and GPUs are inadequate for efficiently deploying GNN models [19]–[21], let alone the more complex subgraph GNNs. Existing GNN accelerators [19]–[22] are optimized for conventional GNNs. However, subgraph GNNs involve numerous independent subgraph operations performed by stacked base GNNs, requiring not only high computing power and memory bandwidth but also the ability to handle subgraph-level parallelism with irregular data accesses. The conventional GNN accelerators, optimized for single-graph processing, struggle with the parallel processing of subgraphs and the accompanying challenges, such as load balancing and massive uneven data movements.

To facilitate the efficient deployment of subgraph GNNs, we propose 3D-SubG, an accelerator that integrates 3D processing-near-memory (PNM) architecture with digital processing-in-memory (PIM) macros. The PNM architecture [23], [24] based on 3D hybrid bonding packaging technology achieves a tight coupling between DRAM and logic dies through numerous metal pillars. It boasts dense I/Os with a density about 110,000/mm² by around 3μm pitch [23], ensuring exceptionally high bandwidth communication be-

*Corresponding Author: Yufei Ma. (yufei.ma@pku.edu.cn)

This work was supported in part by National Key R&D Program of China (No. 2023YFB4404603), in part by National Natural Science Foundation of China (No. 92164301, 62204003, 62225401, and 61927901).

tween the two dies. By deploying subgraph GNNs using the 3D PNM architecture, 3D-SubG not only benefits from extremely high off-chip bandwidth but also leverages the parallel processing capabilities of multiple distributed logic-to-DRAM (L2D) blocks. Each logic block can independently interact with its corresponding DRAM block to fully exploit the inherent parallelism of subgraphs. Furthermore, we employ PIM macros [25], [26] on the logic die to execute matrix multiplications (MMs) during the combination stage of base GNNs, enhancing the computation density and minimizing weight (W) movements. In addition, the aggregation in base GNNs, formulated as a substantial MM between an ultra-sparse adjacency matrix A and a comparatively sparse feature matrix X, is remarkably irregular and sparse. This makes it more suited for SIMD-based engines, which are designed to flexibly handle such aggregation stage and other vector operations. The contributions are summarized as follows:

- Through comprehensive analysis and profiling, we propose 3D-SubG, a 3D stacked hybrid processing near/in-memory accelerator tailored for subgraph GNNs.
- PIM macros with fixed and regular computation pattern can hardly exploit sparsity in base GNNs. Thus, we employ a Digital PIM (DPIM) macro featuring flexible dataflow and propose bit-level non-zero gathering technique to further leverage data sparsity.
- To efficiently manage the varied workloads of subgraphs, we propose a greedy-based subgraph mapping algorithm, which mitigates workload imbalances among L2D blocks during the subgraph phase.
- The final global pooling step involves aggregating information across all subgraphs, which can lead to extensive inter-block data movement. We introduce a distributed pooling approach, which reduces the substantial inter-block data transfers and alleviates inter-block load imbalance.
- We have implemented 3D-SubG using 22nm technology. In comparison to RTX 3090Ti GPU, 3D-SubG demonstrates an average performace improvement of 146.11×, an area efficiency improvement of 934.18×, and an energy efficiency improvement of 1171.80×.

II. Basis of Subgraph GNNs

Fig. 1 illustrates the execution process of Subgraph Graph Neural Network (Subgraph GNN) models. After extracting the subgraphs from the initial graph based on the k-hop neighbors of the root vertices, the subgraph GNNs operate two phases: subgraph phase and global phase.

Subgraph phase comprises equivariant message-passing and intra-subgraph pooling. Stacked equivariant message-passing layers consist of base GNNs for subgraphs, which serve as the primary components in subgraph GNNs. The t-th iteration of the base GNN process within the k-hop subgraph G_v^k of vertex v can be expressed as follows:

$$h_v^t = \sigma^{(t)}\left(h_v^{(t-1)}, \sum_{w \in \mathcal{N}^1_{G_v^k}(v)} h_w^{(t-1)}\right) \tag{1}$$

TABLE I
NOTATIONS OF SUBGRAPH GNNS

Notation	Meaning	Notation	Meaning
$\mathbf{G}$	graph $\mathbf{G} = (\mathcal{V}, \mathcal{E})$	$\mathcal{V}$	vertices of $\mathbf{G}$
$N_G^k(v)$	k-hop neighbors of v in G	v	vertex of $\mathbf{G}$
G_v^k	k-hop subgraph of v	$\mathcal{E}$	edges of $\mathbf{G}$
h_v	feature of vertex v	$h_{\mathbf{G}}$	representation of $\mathbf{G}$

TABLE II
DATA VOLUME OF ORIGINAL GRAPHS AND SUBGRAPHS.

Dataset	Num. Graphs	Sum Vertices	Num. Subgraphs	Sum Vertices (Subgraph)			
				1-hop	2-hop	3-hop	4-hop
MUTAG	188	3371	3371	10813	21669	33181	43385
BZR	405	14479	14479	45549	97819	164545	234879
ENZYMES	600	19578	19578	94144	193130	289884	375652
PROTEINS	1113	43471	43471	205559	435963	701931	979121

where $\sigma^{(t)}$ is an arbitrary continuous function, h_v^t is the feature of vertex v at t-th iteration and $\mathcal{N}^1_{G_v^k}(v)$ denotes the 1-hop neighbors of v in its k-hop subgraph. Intra-subgraph pooling involves pooling the vertex features within each subgraph to derive the corresponding subgraph representation.

Global phase generates the final graph representation through inter-subgraph pooling. Specifically, the pooling is executed across all subgraphs to combine their representations into the final graph representation. This process can be expressed as:

$$h_{\mathbf{G}} = P^G\left(\left\{h_{G_v^k} \mid v \in \mathbf{G}\right\}\right) \tag{2}$$

Here, P^G represents the graph pooling, $h_{G_v^k}$ is the subgraph representation and $h_{\mathbf{G}}$ is the final graph representation.

III. Architecture and Mapping of 3D-SubG

A. Overview of 3D-SubG Hardware Architecture

The overall architecture of 3D-SubG is depicted in Fig. 2. It consists of a DRAM die featuring a 6 × 6 block configuration, with a logic die densely integrated beneath it using uniformly distributed copper metal pads at the bond interface. Similarly, the logic die is segmented into 6 × 6 blocks, constrained by packaging technology. Each logic block is required to be vertically aligned with its respective DRAM block, enabling direct high-bandwidth data exchanges within this logic-to-DRAM (L2D) block. Additionally, the logic blocks are interconnected through an on-chip 2D mesh network, facilitating efficient data communication among all L2D blocks.

A DPIM-based GNN engine is implemented on the logic die within a L2D block to deliver efficient parallel computing capabilities. Each GNN engine is equipped with eight DPIM macros, segmented into four combination lanes for MM operations required in GNN combination calculations. The combination calculations are characterized by their regular and static memory access patterns suitable for DPIM macros. The engine also includes SIMD cores for handling irregular and dynamic memory accesses in aggregation tasks and other vector computations, such as pooling. To harness the inherent

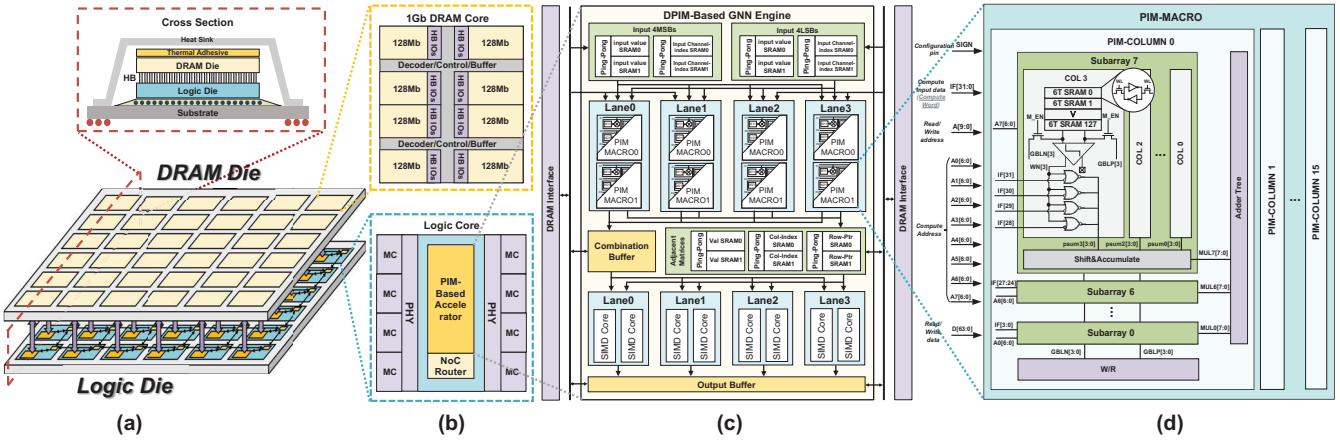

Fig. 2. An overview of the proposed 3D-SubG architecture: (a) the hybrid bonding PNM architecture, (b) a logic-to-DRAM block with a DRAM block and a logic block, (c) micro-architecture of DPIM-based GNN engine inside a logic block, and (d) the micro-architecture of DPIM.

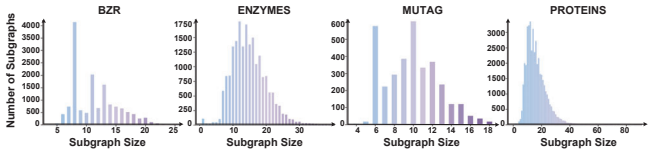

Fig. 3. The count of subgraphs (hop=3) of specific sizes across four datasets reveals an uneven distribution of subgraph sizes.

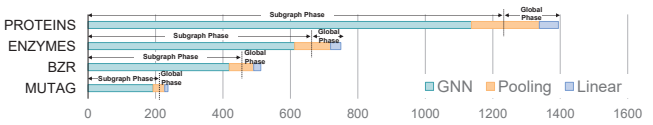

Fig. 4. Runtime breakdown of a subgraph GNN (NGNN) on four different datasets with over 95% attributed to base GNN and pooling.

sparsity of GNNs, a specialized channel-independent DPIM is used in conjunction with our bit-level non-zero (NZ) value gathering method depicted in Fig. 5. For aggregation, the Compressed Sparse Row (CSR) format is used for the adjacency matrix A with 4 flexible SIMD lanes performing sparse matrix multiplication to enhance efficiency in sparse data processing.

During the subgraph processing phase of subgraph GNN, workloads are distributed to L2D blocks at the granularity of subgraphs. This mapping strategy maximizes the parallel execution of subgraphs by leveraging the spatially distributed nature of L2D blocks. Specifically, the 6 × 6 array of L2D blocks enables concurrent processing of up to 36 subgraphs. Once the subgraph phase is completed, the central block collects the computational results from its neighboring blocks and proceeds to execute the final global pooling phase.

B. Sparse-Aware DPIM with Bit-Level Non-Zero Gathering

The primary operator of subgraph GNN is the base GNN, as depicted in Fig. 4. The input matrix X exhibits high sparsity, as evidenced in Table III. Traditional PIM macros, constrained by their rigid dataflow, often find it challenging to effectively

TABLE III
SPARSITY STATISTICS OF THE GNN PART IN NGNN [14]

Matrix		BZR	MUTAG	ENZYMES	PROTEINS
A		0.954	0.882	0.914	0.928
X	Element-wise	0.8562	0.4039	0.2017	0.1906
	4 MSBs	0.8945	0.4696	0.3570	0.2896
	4 LSBs	0.8739	0.5302	0.3190	0.3241
W		0.103	0.0167	0.0653	0.0341

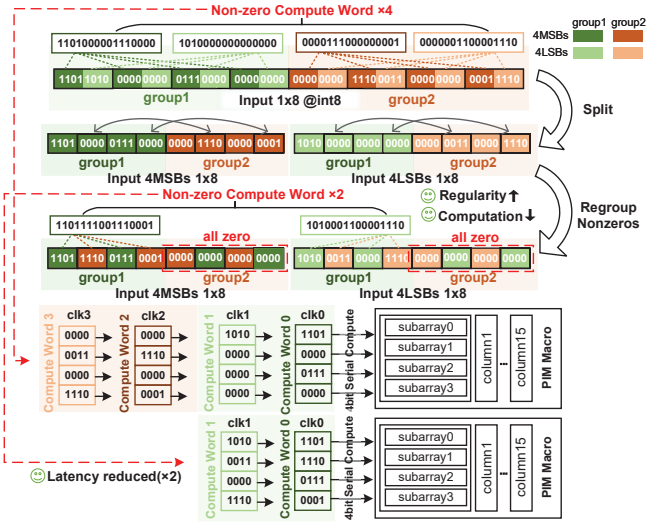

Fig. 5. The proposed bit-level non-zero gathering method for DPIM.

manage sparse data. As illustrated in Table III, among the two matrices involved in the combination stage, weight matrix W has very low sparsity and can be treated as dense, whereas input X displays significantly higher sparsity. Therefore, a weight-stationary DPIM can directly utilize W without additional optimization. The primary focus of optimization efforts should be on how to efficiently exploit the sparsity of X.

This work introduces a channel-independent digital PIM (DPIM) macro for sparse combination computations. Conven-

tional DPIMs are constrained by their reliance on a single subarray address signal, limiting them to selecting weights only from the same position within each subarray. Our DPIM macro, however, allows each input channel to have its unique subarray address signal. This enables concurrent feeding of data from multiple input channels into the DPIM macro for multiplication with different weights stored at various addresses within the subarrays. This design provides an opportunity to leverage sparsity by exchanging input data within the same input channel across different compute words. Different input channels can independently select weights from distinct subarray addresses, ensuring computational accuracy after data exchange. For clarity, we use the term "compute word" to describe the input data that a DPIM macro can process within one clock cycle, analogous to the "word" in SRAM, which denotes the data that can be written or read in a single clock cycle. Furthermore, our DPIM macro employs a 4-bit serial computing approach instead of the conventional 1-bit serial computing, significantly boosting its computing capability.

The bit-level non-zero (NZ) gathering method we propose is illustrated in Fig. 5. The principle of this method involves exchanging data among different compute words within the same input channel. This process gathers non-zero values into specific compute words, thereby increasing the number of all-zero compute words that can be bypassed during computation. It is important to note that compute words composed of different bits from the same data cannot be exchanged, as this would disrupt the proper functioning of the accumulator. Fig. 5 illustrates this method using a DPIM macro with a 4×4 compute word as an example. Before optimization, all four compute words need to be computed. However, after applying NZ gathering, only two compute words remain for processing. While element-level NZ gathering is effective, Table III demonstrates that higher bit-level sparsity makes bit-level NZ gathering even more advantageous, as will be further evidenced in the evaluation results.

C. Load-Balanced Mapping of Subgraphs onto L2D Blocks

In the subgraph phase, the workload is allocated to the L2D blocks at the subgraph granularity. The significant size differences among subgraphs, as presented in Fig. 3, can lead to an imbalanced workload across blocks during parallel computation. To address this issue, we introduce a balanced mapping strategy, as shown in Algorithm 1. It is based on the assumption that the workload of a subgraph generally correlates positively with its vertex count, although the irregular sparsity of the subgraph also plays a role. To estimate the workload of a subgraph, we consider its vertex count and employ a greedy algorithm for subgraph assignment. Initially, we sort the subgraphs in descending order based on their vertex counts. Subsequently, we sequentially assign each subgraph to a target L2D block. The target block is determined based on the number of tasks already assigned to each block. The block with the fewest tasks is the target block. After each subgraph assignment, the record of the block with the fewest tasks is updated, ensuring next subgraph to be assigned in a load-balanced manner.

Algorithm 1: Load-Balanced Mapping of Subgraphs

Input: Subgraph Array $SubG_{array}$, Adjacency matrices array A_{array}, L2D Block Array $Block_{array}$, Gragh Array G_{array}
Output: Block Workload Array BW_{array}

for *each graph* $g \in G_{array}$ **do**
  Update $SubG_{array}$, A_{array}
  for *each subgraph* $subg \in SubG_{array}$ **do**
    $Size[subg] = Number\ of\ rows\ in\ A_{array}[subg]$
  end
  Sort $SubG_{array}$ by $Size$ in descending order
  $Minblock$: the block with the fewest allocated workloads
  for *each subgraph* $subg \in SubG_{array}$ **do**
    for *each block* $b \in Block_{array}$ **do**
      if $BW_{array}[b] < BW_{array}[Minblock]$ **then**
        $Minblock = b$
      else
        Continue
      end
    end
    $BW_{array}[Minblock] = BW_{array}[Minblock] + Size[subg]$
  end
end

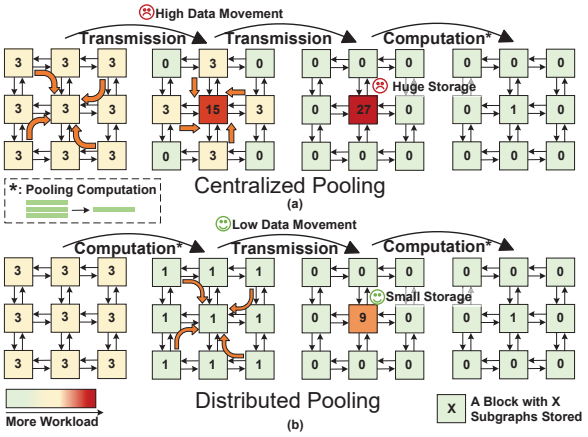

Fig. 6. (a) Centralized global pooling lead to massive data tranfer and imbalanced computation. (b) The proposed distributed pooling method reduces the inter-block data transfer and alleviates workload imbalance.

D. Distributed Global Pooling Approach

Although pooling is often overlooked, global pooling in subgraph GNNs involves significant inter-block data movement, making its optimization crucial. During the subgraph phase, subgraphs are distributed to different L2D blocks. Global pooling requires aggregating computational data from all subgraphs. The most straightforward approach is to use NoC to transmit all subgraph data to a central block, which then performs the final pooling. However, this approach incurs an $O(n)$ data transmission cost through the NoC, where n is the number of subgraphs. The Table II demonstrates that the

number of subgraphs is exceptionally large and continues to increase with the addition of hops. Furthermore, as illustrated in the Fig. 6(a), this method concentrates all computational loads on the central block, resulting in a highly unbalanced load distribution.

Considering the characteristics of pooling, local pooling can be performed within a block first, followed by global pooling. After local pooling, the data volume of each block is reduced to that of a single subgraph. Consequently, the total amount of data that needs to be transferred shifts from being proportional to number of subgraphs to being proportional to number of blocks, thereby significantly mitigating load imbalance and reducing data movement overhead, as shown in Fig. 6(b).

IV. Evaluation

A. Experimental Setup

Architecture Configuration. The specification of 3D-SubG is shown in Table IV, which features a configuration of 6×6 L2D blocks. Every logic block is equipped with eight PIM macros, each comprising a 1024×64-bit SRAM cell array. These SRAM cells are organized into an 8×16 subarray structure, capable of processing 8×16 INT8-by-INT4 operations every two clock cycles. Additionally, each logic block incorporates a total of 276 kB ping-pong SRAM buffers. The NoC on logic die employs a mesh topology with a 64-bit flit width. The DRAM specifications are based on previous research [24] and appropriately scaled to align with this work. The DRAM blocks are arranged in a 6×6 configuration, mirroring the layout of logic blocks. Each DRAM block has a capacity of 16 MB and provides a die-to-die bandwidth of 4.8 GB/s.

Hardware Implementation. The DPIM macro on logic die is evaluated using Cadence Virtuoso for area and power consumption. The on-chip SRAM is generated with ARM Artisan SRAM Compiler, which also supplies data on area and power. The remaining digital components are synthesized using Cadence Genus. The logic die is synthesized using commercial 22nm process, operating at clock frequency of 200 MHz and voltage of 0.8V. The energy cost of DRAM access is estimated as 0.88 pJ/bit [25]. The energy cost of NoC is estimated as 1.1 pJ/bit/hop [27]. Moreover, a C++ simulator is developed to assess the performance, data transfer rates, and utilization of the overall 3D-SubG system.

Benchmarks and Datasets. To comprehensively evaluate 3D-SubG, we deployed and evaluated the Nested GNN (NGNN) [14] on four widely-used datasets in the subgraph GNN domain: BZR [28], MUTAG [29], ENZYMES [30], and PROTEINS [31], all of which belong to TU datasets [32]. NGNN adheres to the general subgraph GNN structure. No specific optimizations were implemented, thereby enabling an unbiased evaluation of 3D-SubG's compatibility with the general subgraph GNN framework. For the model configuration, we adopted the setup outlined in [14], utilizing NGNN with GCN as the base GNN, a hop value of 3, and a hidden dimension of 32. We trained the floating-point model and then quantized it, where the adjacency matrix A and weights W were quantized to INT4, while the graph representation matrix X and model outputs were quantized to INT8. We demonstrated that this quantization scheme has a negligible impact on model accuracy across all four datasets examined, as shown in Table V.

TABLE IV
Specifications of Logic Die and DRAM Die

Logic Die	Specification
Area	98.22 mm^2
Technology	22nm
Block	6×6 blocks
DPIM	12.8GOPS/macro (int8 mul int4)
SRAM Capacity	276kB + 64kB(DPIM) / block
NoC	mesh, 64-bit flit width
Frequency	200MHz
Voltage	0.8V

DRAM Die	Specification
Area	98.22 mm^2 (DRAM 72 mm^2 + Peri 26.22mm^2)
Technology	25nm
Capacity	16MB/block
Bandwidth	4.8GB/s / block
Frequency	150MHz
Voltage	1.1V

TABLE V
Quantization precision testing

Dataset	Claimed in [14] (%)	Floating Point[1](%)	Quantized[1](%)	Accuracy Drop (%)
BZR	–	79.5±3.9	79.0±3.3	0.6
MUTAG	82.9 ± 11.1	81.1 ± 8.3	81.1 ± 8.3	0.0
ENZYMES	31.2 ± 6.7	34.2 ± 12.5	33.8 ± 12.1	1.3
PROTEINS	73.3 ± 4.0	71.8 ± 5.7	71.8 ± 5.7	0.0

[1] The training process employed the 10-fold cross-validation method.

Baselines. To the best of our knowledge, 3D-SubG is the pioneering accelerator tailored for subgraph GNNs. For fairness, we refrain from comparing it with conventional GNN accelerators, whose target task is traditional GNNs. To validate the efficacy of the proposed techniques, we utilize an unoptimized variant of 3D-SubG as the benchmark for comparison. Lastly, we also conducted a comparative analysis of latency, power consumption, and area efficiency with an NVIDIA RTX 3090Ti utilizing PyTorch Geometric.

B. Evaluation Results

Efficiency Evaluation for Bit-Level Non-Zero Gathering. As in Fig. 7, bit-level NZ gathering achieves an average reduction of 2.5×, 1.5×, 1.53×, and 1.93× compute words on BZR, MUTAG, ENZYMES, and PROTEINS, respectively, with the reduction directly proportional to DPIM's computation time. Additionally, we compared bit-level versus element-level NZ gathering, observing reductions of 1.38×, 1.08×, 1.13×, and 1.12× across the datasets. The technique's effectiveness depends on data sparsity and distribution, with BZR showing the greatest improvement due to its highest sparsity as in Tabel III.

Efficiency Evaluation for Load-Balanced Subgraph Mapping. The benefits of load-balanced subgraph mapping are shown in Fig. 8. On the BZR, MUTAG, ENZYMES, and

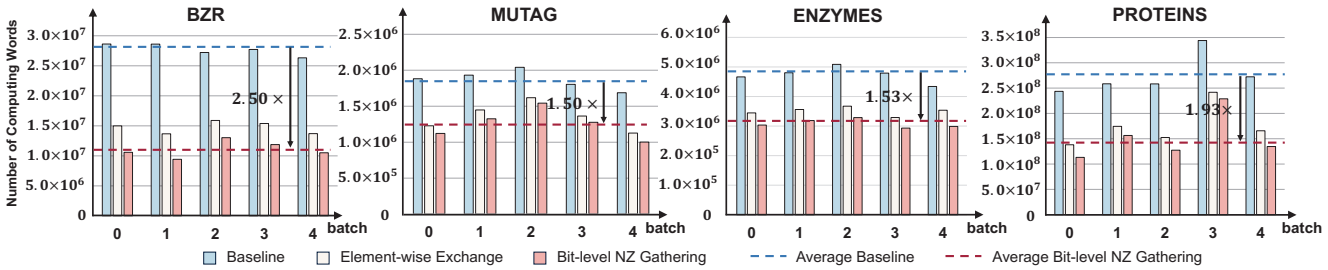

Fig. 7. Number of compute words for DPIM on four datasets.

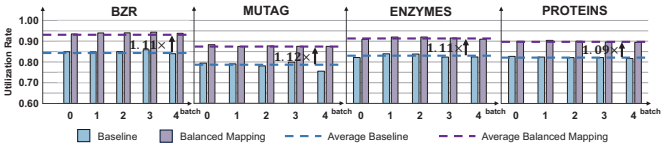

Fig. 8. Average utilization rate of L2D blocks in the subgraph phase.

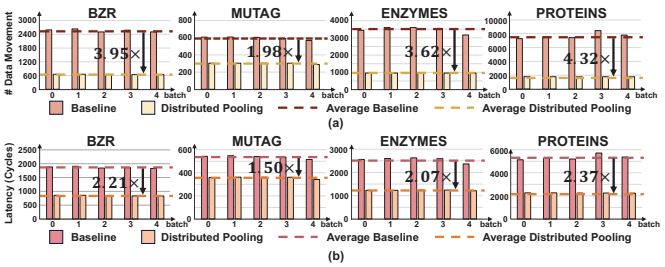

Fig. 9. (a) Amount of data transferred on NoC during the global phase; (b) The latency of global pooling.

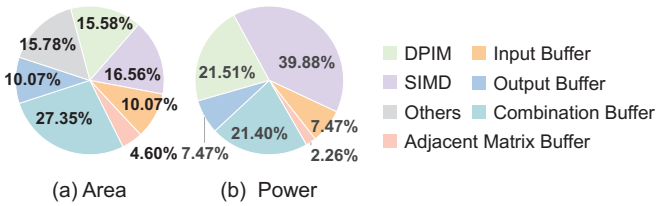

Fig. 10. Area and power breakdown of logic die.

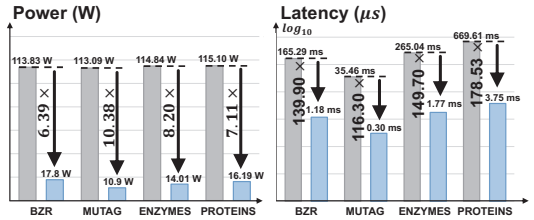

Fig. 11. Comparison results of 3D-SubG with RTX 3090Ti GPU.

PROTEINS datasets, this technique improved the L2D block utilization by 1.11×, 1.12×, 1.11×, and 1.09×, respectively, increasing the overall utilization from around 80% to approximately 90%.

Efficiency Evaluation for Distributed Global Pooling. Distributed global pooling theoretically diminishes the data transfer required for global pooling from $O(n)$ to $O(m)$, where n and m denote the number of subgraphs and L2D blocks, respectively. Experiments confirm this, indicating that on the BZR, MUTAG, ENZYMES, and PROTEINS datasets, distributed pooling attains reductions in inter-block data movement by 3.95×, 1.98×, 3.62×, and 4.32×, respectively, as shown in Fig. 9. By distributing the computational load across all blocks and markedly decreasing data transfer, the total latency of global pooling is also decreased, achieving reductions of 2.21×, 1.50×, 2.07×, and 2.37× on four datasets.

Area and Power Overhead. The implementation results show that the total area of the logic die is 98.22mm^2, with a total power consumption of 4.17W. The area and power breakdowns are shown in Fig. 10.

Overall Performance. We compare 3D-SubG with RTX 3090Ti GPU using PyTorch Geometric, with results presented in Fig. 11. This comparison includes power consumption related to die-to-die and NoC data transmission, as well as DRAM static power consumption. Across four datasets, 3D-SubG outperforms GPU in latency performance by 139.90×, 116.30×, 149.70×, and 178.53×, respectively. Simultaneously, it achieved significant reductions in power consumption by 6.39×, 10.38×, 8.20×, and 7.11×. On average, 3D-SubG exhibited 146.11× improvement in performance, 1171.80× increase in energy efficiency, and 934.18× enhancement in area efficiency compared to RTX 3090Ti GPU.

V. Conclusion

We develop a 3D stacked accelerator, named 3D-SubG, tailored for subgraph GNNs. 3D-SubG is built upon a processing-near-DRAM architecture enabled by hybrid bonding packaging to achieve parallel memory accesses, where the DRAM die is 3D stacked with logic die equipped with PIM macros. For the subgraph phase and global phase in subgraph GNNs, we have proposed load-balanced subgraph mapping and distributed global pooling strategies, respectively. To efficiently manage sparse computations to benefit PIM, we introduced a bit-level non-zero gathering method. On average, 3D-SubG outperforms RTX 3090Ti GPU by 146.11× in performance, 934.18× in area efficiency, and 1171.80× in energy efficiency.

REFERENCES

[1] Z. Ying, J. You, C. Morris, X. Ren, W. Hamilton, and J. Leskovec, "Hierarchical graph representation learning with differentiable pooling," *NeurIPS*, vol. 31, 2018.

[2] Y. Li, C. Gu, T. Dullien, O. Vinyals, and P. Kohli, "Graph matching networks for learning the similarity of graph structured objects," in *ICML*. PMLR, 2019, pp. 3835–3845.

[3] R. Ying, R. He, K. Chen, P. Eksombatchai, W. L. Hamilton, and J. Leskovec, "Graph convolutional neural networks for web-scale recommender systems," in *KDD*, 2018, pp. 974–983.

[4] A. Fout, J. Byrd, B. Shariat, and A. Ben-Hur, "Protein interface prediction using graph convolutional networks," *NeurIPS*, vol. 30, 2017.

[5] K. Do, T. Tran, and S. Venkatesh, "Graph transformation policy network for chemical reaction prediction," in *KDD*, 2019, pp. 750–760.

[6] M. Haggai, B. Heli, S. Nadav, and L. Yaron, "Invariant and equivariant graph networks," in *ICLR*, 2019.

[7] X. Keyulu, H. Weihua, L. Jure, and J. Stefanie, "How powerful are graph neural networks?" in *ICLR*, 2019.

[8] C. Morris, M. Ritzert, M. Fey, W. L. Hamilton, J. E. Lenssen, G. Rattan, and M. Grohe, "Weisfeiler and leman go neural: Higher-order graph neural networks," in *AAAI*, vol. 33, no. 01, 2019, pp. 4602–4609.

[9] Z. Lingxiao, J. Wei, A. Leman, and S. Neil, "From stars to subgraphs: Uplifting any gnn with local structure awareness," in *ICLR*, 2022.

[10] A. Leman and B. Weisfeiler, "A reduction of a graph to a canonical form and an algebra arising during this reduction," vol. 2, no. 9, 1968, pp. 12–16.

[11] Z. Chen, L. Chen, S. Villar, and J. Bruna, "Can graph neural networks count substructures?" *NeurIPS*, vol. 33, 2020.

[12] V. Arvind, F. Fuhlbruck, J. Kobler, and O. Verbitsky, "On weisfeiler-leman invariance: Subgraph counts and related graph properties," *Journal of Computer and System Sciences*, vol. 113, pp. 42–59, 2020.

[13] D. C. Elton, Z. Boukouvalas, M. D. Fuge, and P. W. Chung, "Deep learning for molecular design—a review of the state of the art," *Molecular Systems Design & Engineering*, vol. 4, no. 4, pp. 828–849, 2019.

[14] M. Zhang and P. Li, "Nested graph neural networks," *NeurIPS*, vol. 34, pp. 15 734–15 747, 2021.

[15] B. Bevilacqua, F. Frasca, D. Lim, B. Srinivasan, C. Cai, G. Balamurugan, M. M. Bronstein, and H. Maron, "Equivariant subgraph aggregation networks," in *ICLR*, 2022.

[16] L. Cotta, C. Morris, and B. Ribeiro, "Reconstruction for powerful graph representations," in *NeurIPS*. Curran Associates Inc., 2024.

[17] J. You, J. M. Gomes-Selman, R. Ying, and J. Leskovec, "Identity-aware graph neural networks," in *Proceedings of the AAAI conference on artificial intelligence*, vol. 35, no. 12, 2021, pp. 10 737–10 745.

[18] F. Frasca, B. Bevilacqua, M. Bronstein, and H. Maron, "Understanding and extending subgraph gnns by rethinking their symmetries," *NeurIPS*, vol. 35, 2022.

[19] M. Yan, L. Deng, X. Hu, L. Liang, Y. Feng, X. Ye, Z. Zhang, D. Fan, and Y. Xie, "Hygcn: A gcn accelerator with hybrid architecture," in *HPCA*, 2020.

[20] T. Geng, A. Li, R. Shi, C. Wu, T. Wang, Y. Li, P. Haghi, A. Tumeo, S. Che, S. Reinhardt *et al.*, "Awb-gcn: A graph convolutional network accelerator with runtime workload rebalancing," in *MICRO*, 2020, pp. 922–936.

[21] R. Hwang, M. Kang, J. Lee, D. Kam, Y. Lee, and M. Rhu, "Grow: A row-stationary sparse-dense gemm accelerator for memory-efficient graph convolutional neural networks," in *HPCA*, 2023, pp. 42–55.

[22] Z. Zhu, F. Li, G. Li, Z. Liu, Z. Mo, Q. Hu, X. Liang, and J. Cheng, "Mega: A memory-efficient gnn accelerator exploiting degree-aware mixed-precision quantization," in *HPCA*, 2024, pp. 124–138.

[23] Z. Yue, H. Wang, J. Fang, J. Deng, G. Lu, F. Tu, R. Guo, Y. Li, Y. Qin, Y. Wang, C. Li, H. Han, S. Wei, Y. Hu, and S. Yin, "Exploiting similarity opportunities of emerging vision ai models on hybrid bonding architecture," in *ISCA*, 2024.

[24] D. Niu, S. Li, Y. Wang, W. Han, Z. Zhang, Y. Guan, T. Guan, F. Sun, F. Xue, L. Duan, Y. Fang, H. Zheng, X. Jiang, S. Wang, F. Zuo, Y. Wang, B. Yu, Q. Ren, and Y. Xie, "184qps/w 64mb/mm23d logic-to-dram hybrid bonding with process-near-memory engine for recommendation system," in *ISSCC*, vol. 65, 2022.

[25] Y. Chi *et al.*, "16.4 an 89tops/w and 16.3 tops/mm2 all-digital sram-based full-precision compute-in memory macro in 22nm for machine-learning edge applications," in *ISSCC*, vol. 64, 2021, pp. 252–254.

[26] A. Guo, X. Si, X. Chen, F. Dong, X. Pu, D. Li, Y. Zhou, L. Ren, Y. Xue, X. Dong *et al.*, "A 28nm 64-kb 31.6-tflops/w digital-domain floating-point-computing-unit and double-bit 6t-sram computing-in-memory macro for floating-point cnns," in *ISSCC*, 2023, pp. 128–130.

[27] J. Wang, H. Du, B. Ding, Q. Xu, S. Chen, and Y. Kang, "Ddam: Data distribution-aware mapping of cnns on processing-in-memory systems," *ACM Trans. Des. Autom. Electron. Syst.*, vol. 28, no. 3, 2023.

[28] J. J. Sutherland, L. A. O'Brien, and D. F. Weaver, "Spline-fitting with a genetic algorithm: A method for developing classification structure-activity relationships," *Journal of chemical information and computer sciences*, vol. 43 6, pp. 1906–1915, 2003.

[29] N. Kriege and P. Mutzel, "Subgraph matching kernels for attributed graphs," in *ICML*, 2012, p. 291–298.

[30] I. Schomburg, A. Chang, C. Ebeling, M. Gremse, C. Heldt, G. Huhn, and D. Schomburg, "Brenda, the enzyme database: updates and major new developments," *Nucleic Acids Research*, vol. 32, no. suppl_1, 01 2004.

[31] K. M. Borgwardt, C. S. Ong, S. Schönauer, S. V. N. Vishwanathan, A. J. Smola, and H.-P. Kriegel, "Protein function prediction via graph kernels," *Bioinformatics*, vol. 21, pp. i47–i56, 2005.

[32] C. Morris, N. M. Kriege, F. Bause, K. Kersting, P. Mutzel, and M. Neumann, "Tudataset: A collection of benchmark datasets for learning with graphs," *ArXiv*, vol. abs/2007.08663, 2020.

Late reaking Results o el Design o T ased ni ied L F S iking euron and P F

ilad Tana ardi asa Wu ang and imanshu Tha liyal
Department of Electrical Engineering and Computer Science, University of Tennessee, Knoxville, TN, USA

2025 62nd ACM/IEEE Design Automation Conference (DAC) | 979-8-3315-0304-8/25/$31.00 ©2025 IEEE | DOI: 10.1109/DAC63849.2025.11132831

stract **Due to t e ig er energ and ard are efficienc of s i ing neural net or s S s co ared to dee neural net or s t e a e attracted a lot of attention o e er t eir securit ust e in estigated gi en t at t e a e access to ri ate and confidential data sicall unclona le functions s are a class of circuits it securit a lications li e de ice aut entication e edded licensing de ice s ecific cr togra ic e generation and anti counterfeiting erefore s can e used to en ance t e securit of S ccordingl in t is a er an M ased I euron as een ro osed e ro osed design can function as ot a I neuron and e results of t e Monte Carlo si ulation s o t at t e ro osed design as etter uni ueness and unifor it alues co ared to its counter arts ese alues for t e ro osed design are and ic are close to t eir ideal alue of lso t e ean alue of S annon entro for t e it res onse of t e ro osed design is ic is close to its ideal alue of**

e wor s Spi ing eural etwor ar ware Securit

TR D T

S iking neural net ork S is a romising alternati e or dee neural net orks S s are e ent dri en net orks in hich the data is encoded into s ikes Data encoding and rocessing alongside eing e ent dri en make S a more energy and hard are e icient choice com ared to D s ccordingly S s are a suita le choice or energy constrained a lications such as oT 1 Due to the a lication o S s in rocessing data they ha e access to the ri ate and sometimes con idential in ormation and data o the user ccordingly the security o these net orks is o great im ortance

Physically unclona le unctions P Fs are a class o circuits or security ur oses They can e used or many a lications such as de ice authentication em edded licensing de ice s eci ic cry togra hic key generation and anti counter eiting 2 These circuits ha e een idely used in many a lications to enhance security n and 4 the authors ro osed euron P F or s iking neural net orks n these a ers the reset duty cycle and eriodicity o s iking neurons and their de endency on rocess ariations ha e een used as the source o entro y and additional circuits are needed to generate the res onse o P F n order to eliminate the need or these additional circuits generate the res onse y the neuron itsel and uni y neuron and P F **in t is a er a agnetic tunnel unction M ased lea integrate fire I s i ing euron as een ro osed** nlike the ro osed design in and 4 rocess ariation o T has een used as the source o entro y The ro osed design can ork as oth a neuron and P F and does not re uire any au iliary circuits to generate the res onse o P F and the res onse is generated y the neuron itsel

PR P S D D S

The ro osed design is sho n in Fig 1 T s ha e een used in the structure o the ro osed design T consists o t o erromagnetic layers se arated y a thin o ide arrier The magnetic orientation o one o these layers is i ed i ed layer and the other one is modi ia le ree layer the magnetic orientation o the ree layer is the same as o osite to the i ed layer T is in arallel anti arallel mode and it sho s relati ely lo high resistance

This ork as su orted in art y the ational Science Foundation R R ard under rant 22 22

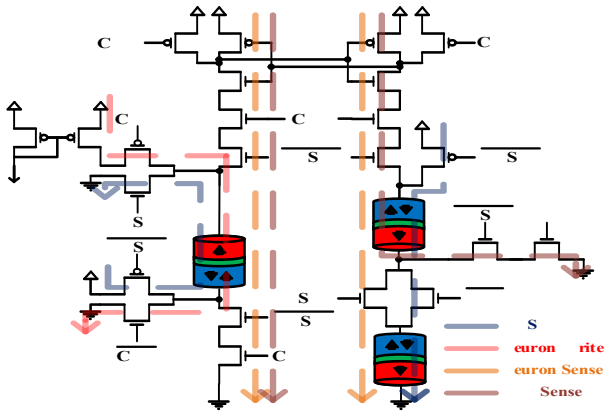

Fig 1 Schematic o the ro osed design The le t T to right and ottom right T are re erred to as L T R T 1 and R T 2 res ecti ely

The ro osed design has t o modes euron and P F ne STT T L T is used on the le t hand side hose de ault state is nti arallel and t o STT T R T 1 or the to one and R T 2 or the ottom one ith a de ault state o arallel ha e een used on the right hand side The critical arameters o transistors and T s are re orted in Section The unctionality o the ro osed design in euron and P F mode is e lained in the ollo ing su sections

A. *Neuron mode functionality*

The stochastic Landau Li shit il ert Slonc e ski e uation hich go erns the dynamic characteristics o magneti ation o STT T is sho n in 1 The recession dam ing and s in trans er tor ue are re resented y the irst second and third terms o the right hand side o 1 lso the mem rane oltage $u(t)$ o a L F neuron im lemented y the ca acitance o the resistance o R $\tau_m = RC$) and the current source o $J(t)$ is calculated using 2 the reset o eration is not descri ed in this e uation y com aring 1 and 2 it can e concluded that y considering recession and dam ing terms as the $-\frac{u(t)}{\tau_m}$ term and s in trans er term as $\frac{J(t)}{C}$ the mem rane otential can e imitated y magneti ation o the ree layer o the STT T 1

$$\frac{d\vec{M}}{dt} = -\gamma\vec{M} \times (\overrightarrow{H_{eff}} + \overrightarrow{H_{th}}) + \frac{\alpha}{M_s}\vec{M} \times \frac{d\vec{M}}{dt} + \frac{\gamma\eta\hbar I}{2eM_s^2 V}\vec{M} \times (\vec{M} \times \vec{P}) \quad 1$$

$$\frac{du}{dt} = -\frac{u(t)}{\tau_m} + \frac{J(t)}{C} \quad 2$$

The euron mode o the ro osed design can e se arated into t o arts The L and R S T signals are 0 in the irst art The in ut current goes through the L and it is co ied y the current mirror This co ied current then goes through the L T trying to change the state o the L T rom anti arallel to arallel n the second art o the euron mode the L signal is set to 1 and the in ut circuit is disconnected The re charged sense am li ier P S on the to com ares its le t and right hand sides ath resistance including the resistance o the L T and the resistance o the R T 1 and R T 2 in series There ore in case the in uts ha e een a le to change the state o the L T the out ut ill e 0 other ise it ill e 1

ter roducing the out ut the R S T signal ill e set to 1 to change the state o T s to their de ault state The corres onding aths or each o these stages ha e een sho n in Fig 1

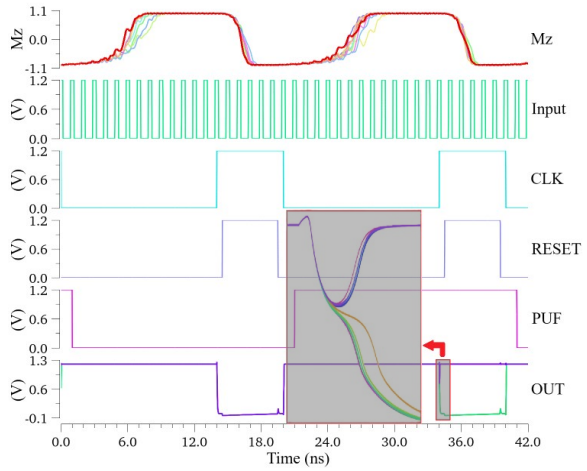

Fig 2 ut ut a e orm o 10 onte arlo simulations o the ro osed euron P F

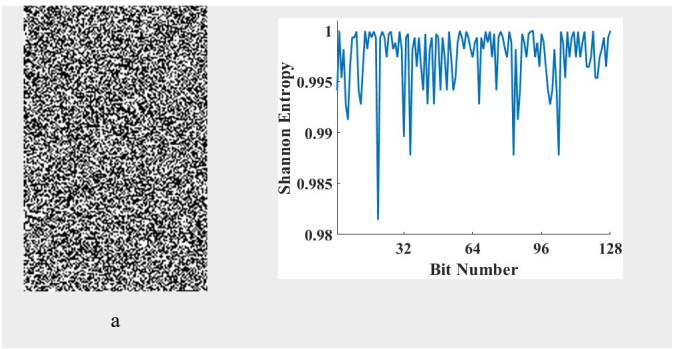

Fig 200 instances of 128-bit PUF response using the proposed design a) Grayscale bitmap, b)Shannon entropy of each bit of the response

B. PUF mode functionality

During P F mode irst the state o the L T is set to arallel hen the L signal is 0 When the L signal rises rom 0 to 1 the R T 2 is disconnected and the alternati e ath containing t o S transistors controlled y $\overline{RESET}$ and P F signal is connected to the ottom o R T 1 n this case R T 1 and L T are in arallel state and they are com ared to each other y P S ccordingly the out ut o the ro osed euron P F is random and de ends on the rocess ariations ter roducing the out ut the reset o eration ill e the same as the euron mode

S L T R S LTS

n this a er SP simulation using adence S ectre has een used or the simulations TS 6 nm S PD and the T model resented in ha e een used or the simulations The gate length and idth o oth S and P S transistors ha e een set to 60nm and 120nm lso the sur ace o ide arrier thickness and ree layer thickness o the T s ha e een set to 0nm 0nm 0 8 nm and 2nm

Fig 2 sho s the out ut a e orm o 10 onte arlo simulations o the ro osed design From 0ns to 21ns the ro osed design is in the euron mode and a ter that it is in the P F mode The a e orm la eled as in Fig 2 sho s the magnetic direction o the L T 1 or anti arallel and 1 or arallel s sho n in Fig 2 hen the ro osed design is in euron mode all the out uts o di erent instances are e ual to each other 14ns to 20ns and hen it is in P F mode due to the rocess ariation the out uts o di erent instances are di erent

n order to in estigate the er ormance o the ro osed design as a P F 200 onte arlo simulations o 128 it P F ha e een er ormed y instantiating 200 128 o the ro osed design The grayscale itma o this simulation is sho n in Fig a ni ueness uni ormity and Shannon entro y as an indicator o randomness ha e een used as er ormance metrics ni ueness indicates ho di erent the P F res onses o di erent instances are hile uni ormity indicates the ratio o ones in res onse to the length o the res onse The ideal alue or oth o these metrics is 0 n the other hand Shannon entro y indicates the randomness o the P F ith the ideal alue o 1 100 The Shannon entro y or each o the res onse its is sho n in Fig The alues o these metrics or the ro osed design are sho n in Ta le The alues o uni ormity and uni ueness o the ro osed design are 4 66 and 0 0 res ecti ely hich are close to their ideal alues The re orted alue o uni ormity or and 4 are 4 4 and 4 4 hich oth ha e higher di erences ith the ideal alue com ared to the ro osed design The uni ueness metric has een only re orted in hich is 48 42 hich has a higher di erence ith the ideal alue com ared to the ro osed design lso the ma imum mean minimum and standard de iation STD o the Shannon entro y o each it o res onse are 1 0 4 0 8 1 and 0 00 1 hich are close to the ideal alue

T L *Performance metrics of the proposed design in PUF mode.*

etric	ni ormity	ni ueness	Shannon ntro y			
			a	ean	in	STD
Pro osed design			1	0 4	0 8 1	0 00 1
	4 4	48 42				
4	4 4					

stands or ot aila le

L S

n this a er an T ased L F s iking euron P F has een ro osed The ro osed design has t o modes euron and P F mode n the neuron mode the dynamic characteristic o the magneti ation o the ree layer o the STT T has een utili ed to imitate the unctionality o a L F neuron This design can e utili ed as a P F in P F mode ithout any additional circuits The P F mode o the ro osed design can e used or security a lications like authentication licensing and anti counter eiting a lications onte arlo simulations ha e een er ormed to e aluate the er ormance o the ro osed design Results sho that the uni ormity uni ueness and Shannon entro y randomness metric o the ro osed design are 4 66 0 0 and 0 4 mean alue res ecti ely hich are close to their ideal alues

R F R S

1 Liu and T hsa a inari ed s iking neural net ork ased on auto reset L F neurons and large signal syna ses using STT T s *Japanese journal of applied physics,* ol 62 no 4 044 01 202

2 R Della Sala D elli ia and Scotti n eiling the True Po er o the Latched Ring scillator or a ni ied P F and TR rchitecture *IEEE Trans. on VLSI Systems vol. 32, no. 12, pp. 2403-2407, Dec. 2024.*

lshamy and Stratigo oulos euron P F Physical nclona le unction ased on a single s iking neuron in *2021 IEEE Symp. On-Line Testing and Robust System Design (IOLTS)* 1 6

4 Takaloo hmadi and hmadi S iking eurons e ntro y Source or Physically nclona le Functions in *2022 IEEE International Symp. on Circuits and Systems (ISCAS)* 1 0 1 4

Wang W hao Deng lein and ha ert Per endicular anisotro y magnetic tunnel junction s itched y s in all assisted s in trans er tor ue *Journal of Physics D: Applied Physics,* ol 48 no 6 201 doi 10 1088 0022 2 48 6 06 001